

UrbanSphere: Simulated Retail Project in SQL

Tools Used: MySQL Workbench, Excel, Word, ChatGPT(for data simulation)

Venkata Sai Varun Vedula

31 July 2025, 3:10 PM

Project Overview:

UrbanSphere is a fictional, U.S.-based omnichannel retail brand that operates across multiple product categories, including Electronics, Fashion, Home, Fitness, and Beauty. The company aims to leverage customer orders, payments, returns, and regional performance data to make strategic decisions. As a business analyst, this project simulates a real-world scenario where I've been tasked with uncovering insights that can help optimize product performance, improve customer experience, and streamline operations.

This project was created using synthetic retail data generated with the assistance of ChatGPT, structured to reflect realistic business challenges and to demonstrate SQL querying and analytical capabilities from a business analyst perspective.

Purpose and Goal of the Project:

The main objective of this project is to demonstrate how SQL can be used to perform end-to-end data analysis in a business setting. Using a relational retail database, I've analyzed customer behavior, payment trends, return patterns, and product performance. The goal is to build familiarity with querying relational databases to extract insights that drive business decisions and to convert theoretical SQL knowledge into practical business analysis.

Database Description & ERD:

The database consists of 6 normalized tables:

1. regions (region_id [PK], region_name, city)
2. customers (customer_id [PK], customer_name, email, signup_date, segment, region_id [FK → regions.region_id])
3. products (product_id [PK], product_name, category, brand, price)
4. orders (order_id [PK], customer_id [FK → customers.customer_id], product_id [FK → products.product_id], order_date, quantity, order_status)
5. payments (payment_id [PK], order_id [FK → orders.order_id], payment_method, discount, final_amount)
6. returns (return_id [PK], order_id [FK → orders.order_id], return_date, return_reason, refund_status)

ERD:

```
regions.region_id -----> customers.region_id
customers.customer_id --> orders.customer_id
products.product_id ----> orders.product_id
orders.order_id -----> payments.order_id
orders.order_id -----> returns.order_id
```

Key SQL Concepts Covered:

1. SELECT, WHERE ORDER BY
2. GROUP BY and aggregate functions (SUM, COUNT, AVG)
3. JOINS (INNER, LEFT)
4. CASE statements
5. Filtering with NULLs and COALESCE
6. Basic subqueries
7. Window functions
8. Data filtering, transformation, and conditional classification
Data manipulation using aliases, rounding, string formatting, and NULL handling

SQL Use Cases from UrbanSphere:

1. What is the total number of orders placed per product category?

```
4  USE UrbanSphere;
5
6  SELECT * FROM orders;
7  SELECT * FROM products;
8
9  SELECT
10     p.category,
11     SUM(o.quantity) AS total_quantity
12  FROM orders AS o
13  JOIN products AS p ON o.product_id = p.product_id
14  GROUP BY p.category;
15
```

Purpose: This query shows how many total items were ordered per product category. It helps in identifying which product categories are in the highest demand.

Output:

category	total_quantity
Electronics	89
Fashion	52
Fitness	41
Home	62
Beauty	62

2. List the top 5 customers by the number of orders they've placed.

```
# Q2 List the top 5 customers by the number of orders they've placed.
```

```
SELECT * FROM orders;
SELECT * FROM customers;

SELECT
    customer_id,
    SUM(quantity) as total_orders
FROM orders
GROUP BY customer_id
ORDER BY SUM(quantity) DESC LIMIT 5;
```

Purpose: This query identifies the most active customers by the total quantity of orders placed, highlighting customer loyalty or bulk buyers.

Output:

customer_id	total_orders
122	19
105	16
134	15
107	15
108	14

3. What is the total revenue collected for each order status?

```
#Q3 What is the total revenue collected for each order status?
```

```
SELECT * FROM orders;
SELECT * FROM payments;

SELECT
    o.order_status,
    SUM(p.final_amount) as total_revenue
FROM orders as o
JOIN payments as p
ON o.order_id = p.order_id
GROUP BY o.order_status;
```

Purpose: This query calculates the total revenue based on each order's status, helping to understand how revenue aligns with fulfillment success or delays.

Output:

order_status	total_revenue
delivered	3688.18
Shipped	4027.27
Delivered	2920.70
Delivered	3734.62
Cancelled	3671.72

4. Which payment method is used most often, excluding NULLs?

```
SELECT
    payment_method,
    COUNT(COALESCE (payment_method, 0)) as count_pm
FROM payments
WHERE payment_method IS NOT NULL
GROUP By payment_method
ORDER BY count_pm DESC;
```

Purpose: This query shows the number of customers in each city, providing insights into geographic concentration of the customer base.

Output:

payment_method	count_pm
COD	34
Gift Card	31
Credit Card	30
PayPal	24

5. Rank customers within each region based on total revenue generated.

```
SELECT
  r.region_name,
  c.customer_id,
  c.customer_name,
  SUM(p.final_amount) AS total_revenue,
  RANK() OVER (PARTITION BY r.region_name ORDER BY SUM(p.final_amount) DESC) AS revenue_rank
FROM customers AS c
JOIN regions AS r ON c.region_id = r.region_id
JOIN orders AS o ON c.customer_id = o.customer_id
JOIN payments AS p ON o.order_id = p.order_id
GROUP BY r.region_name, c.customer_id, c.customer_name
ORDER BY r.region_name, revenue_rank;
```

Purpose: This query identifies which payment method is used most often, offering insights into user preferences and aiding payment system decisions.

Output:

region_name	customer_id	customer_name	total_revenue	revenue_rank
East Coast	129	Andrew Newton	868.06	1
East Coast	112	Zachary Riley	684.19	2
East Coast	120	Brenda Beck	563.24	3
East Coast	125	Tammy Meyer	425.46	4
East Coast	104	Ms. Monica Taylor	138.84	5
Midwest	122	Jose Hall	1000.99	1
Midwest	107	Bethany Bowen	870.97	2
Midwest	126	Margaret Rogers	619.17	3
Midwest	116	Dr. Charles Hunt	528.01	4
Midwest	130	Jared Miles	365.98	5
Midwest	133	Jeremiah Sanders	262.54	6
Midwest	119	Brian Todd	241.16	7
Midwest	102	Kristi Hays	144.84	8
Pacific Northwest	105	William Kim	1246.48	1
Pacific Northwest	134	Denise Sandoval	1183.30	2
Pacific Northwest	118	Nathan Hill	640.96	3
Pacific Northwest	127	Deborah Brewer	307.23	4
Pacific Northwest	115	Douglas Robinson	303.74	5
Pacific Northwest	110	Jillian Morse	258.64	6
South	135	Carlos Munoz	765.17	1
South	117	Denise Dickerson	639.88	2
South	114	Lauren Price	491.32	3
South	103	Aaron Lewis	421.41	4
South	123	Amber Henderson	421.07	5
South	132	Annette Graham	180.69	6
West Coast	108	Nicholas Mcpherson	992.40	1
West Coast	131	Brianna Lewis	739.54	2
West Coast	128	Samuel Schultz	646.33	3
West Coast	109	Robert Whitehead	639.42	4
West Coast	111	Timothy Logan	497.47	5
West Coast	101	Clayton Yates	439.06	6
West Coast	113	Samuel Leonard	274.53	7
West Coast	124	Michael Martin	240.40	8
West Coast	121	Theresa Johnson	NULL	9

6. Which customer has generated the most total revenue?

```
SELECT
    c.customer_name,
    SUM(p.final_amount) as total_revenue
FROM customers as c
JOIN orders as o
ON c.customer_id = o.customer_id
JOIN payments as p
ON o.order_id = p.order_id
GROUP BY c.customer_name
ORDER BY SUM(p.final_amount) DESC LIMIT 1;
```

Purpose: This query filters for orders where final payment amounts are missing, helping identify potential data quality issues or failed transactions.

Output:

customer_name	total_revenue
William Kim	1246.48

7. Show the average final amount paid for each payment method.

```
SELECT
    payment_method,
    COALESCE(AVG(final_amount), 0) as avg_amount
FROM payments
WHERE payment_method IS NOT NULL
GROUP BY payment_method;
```

Purpose: This query ranks cities based on total revenue generated, helping determine top-performing geographic regions for strategic planning.

Output:

payment_method	avg_amount
Gift Card	142.316923
PayPal	121.151429
COD	131.712903
Credit Card	141.423214

8. List all orders that have NULL values in either payment_method or final_amount.

```
SELECT
    payment_id,
    payment_method,
    final_amount
FROM payments
WHERE payment_method IS NULL OR final_amount IS NULL;
```

Purpose: This query identifies payments where either the method or amount is missing, useful for data validation and cleaning.

(Output: Too Long to be displayed.)

9. Which customers have placed only one order?

```
SELECT
    c.customer_id,
    COUNT(o.order_id)
FROM customers AS c
JOIN orders as o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id
HAVING COUNT(o.order_id) = 1;
```

Purpose: This query identifies customers who have placed only a single order. It is useful for analyzing one-time buyers and helps in customer retention strategy or re-engagement campaigns.

Output: (Note - No customer who has placed only single order)

customer_id	COUNT(o.order_id)
-------------	-------------------

10. Identify customers who placed orders in both 2024 and 2025.

```
SELECT
    o.customer_id,
    c.customer_name
FROM orders AS o
JOIN customers AS c
ON c.customer_id = o.customer_id
WHERE YEAR(o.order_date) IN (2024,2025)
GROUP BY o.customer_id, c.customer_name
HAVING COUNT(DISTINCT YEAR(o.order_date)) = 2;
```

Purpose: This query finds customers who made purchases in both 2024 and 2025, helping analyze customer retention over time.

Output:

customer_id	customer_name
103	Aaron Lewis
104	Ms. Monica Taylor
105	William Kim
107	Bethany Bowen
108	Nicholas Mcpherson
109	Robert Whitehead
111	Timothy Logan
113	Samuel Leonard
116	Dr. Charles Hunt
117	Denise Dickerson
118	Nathan Hill
119	Brian Todd
121	Theresa Johnson
122	Jose Hall
123	Amber Henderson
124	Michael Martin
125	Tammy Meyer
126	Margaret Rogers
127	Deborah Brewer
129	Andrew Newton
131	Brianna Lewis
133	Jeremiah Sanders
135	Carlos Munoz

11. List product categories that had at least one order but zero returns.

```
SELECT DISTINCT
    p.category
FROM products p
JOIN orders o ON p.product_id = o.product_id
LEFT JOIN returns r ON o.order_id = r.order_id
WHERE r.return_id IS NULL;
```

Purpose: This query identifies categories with orders but zero returns, useful for spotting high-performing, low-risk products.

Output:

category
Electronics
Fashion
Fitness
Home
Beauty

12. Display all products with a label based on price using CASE (Budget, Standard, Premium).

```
SELECT
    *,
    CASE
        WHEN price <= 30 THEN 'Budget'
        WHEN price BETWEEN 30 AND 70 THEN 'Standard'
        WHEN price > 70 THEN 'Premium'
    END AS labels
FROM products
ORDER BY price DESC;
```

Purpose: This query classifies products into price bands using CASE statements. It supports pricing strategy analysis and segmentation in marketing or retail planning.

Output:

product_id	product_name	category	brand	price	labels
218	Almost Set	Home	SoundPro	113.03	Premium
207	Dog Plus	Home	SpeedTek	105.62	Premium
201	Purpose Set	Electronics	ZenTech	91.57	Premium
202	Brother Max	Fashion	UrbanStyle	91.01	Premium
221	Discover Lite	Fitness	UrbanStyle	85.31	Premium
214	Offer Max	Electronics	ZenTech	82.74	Premium
224	Play Plus	Home	UrbanStyle	82.10	Premium
219	All Set	Electronics	FlexiGear	79.85	Premium
203	Ago Set	Fitness	ZenTech	74.95	Premium
205	Face Plus	Fitness	ZenTech	71.74	Premium
212	Sit Pro	Beauty	ZenTech	70.72	Premium
220	Better Plus	Fashion	UrbanStyle	60.85	Standard
211	Require Lite	Electronics	SoundPro	60.54	Standard
216	Performance Pro	Beauty	AromaCo	59.88	Standard
213	Wait Set	Fitness	SpeedTek	49.78	Standard
217	Knowledge Lite	Fashion	AromaCo	49.08	Standard
210	Blue Pro	Home	SpeedTek	47.84	Standard
208	Chair Max	Beauty	AromaCo	40.57	Standard
225	Agreement XL	Beauty	UrbanStyle	39.14	Standard
215	Begin Lite	Electronics	SpeedTek	35.61	Standard
206	Election Set	Fitness	GlowUp	34.25	Standard
204	Site Pro	Electronics	ZenTech	34.05	Standard
209	Since Max	Home	ZenTech	20.20	Budget
223	Mouth Lite	Beauty	AromaCo	17.28	Budget
222	Early Pro	Fashion	SpeedTek	13.53	Budget

13. Count how many customers fall into each segment.

```
SELECT
    COUNT(customer_id) as number_of_cust,
    segment
FROM customers
GROUP BY segment;
```

Purpose: This query reveals the distribution of customers across defined segments (e.g., Gold, Silver). It's useful for understanding customer base composition and tailoring segment-specific strategies.

Output:

number_of_cust	segment
11	Gold
11	Silver
7	Platinum
6	NULL

14. List the product with the highest single final payment amount in one order.

```
SELECT
    p.product_name,
    pa.order_id,
    pa.final_amount
FROM payments pa
JOIN orders o ON pa.order_id = o.order_id
JOIN products p ON o.product_id = p.product_id
ORDER BY pa.final_amount DESC
LIMIT 1;
```

Purpose: This query highlights the top-grossing product per transaction, helping uncover high-ticket items that may drive premium revenue.

Output:

product_name	order_id	final_amount
Require Lite	394	247.13

15. List customers who placed orders but never had any returns.

```
SELECT DISTINCT
    c.customer_name
FROM customers as c
JOIN orders as o
ON c.customer_id = o.customer_id
WHERE o.order_id NOT IN (
    SELECT order_id FROM returns
);
```

Purpose: This query isolates customers with a clean transaction history—those who never returned a product. It's essential for recognizing loyal or satisfied customers and targeting retention efforts.

Output:

customer_name
Clayton Yates
Kristi Hays
Aaron Lewis
Ms. Monica Taylor
William Kim
Bethany Bowen
Nicholas Mcpherson
Robert Whitehead
Jillian Morse
Timothy Logan
Zachary Riley
Samuel Leonard
Lauren Price
Dr. Charles Hunt
Denise Dickerson
Nathan Hill
Brian Todd
Brenda Beck
Theresa Johnson
Jose Hall
Amber Henderson
Michael Martin
Tammy Meyer
Margaret Rogers
Deborah Brewer
Samuel Schultz
Andrew Newton
Jared Miles
Brianna Lewis
Annette Graham
Jeremiah Sanders
Denise Sandoval
Carlos Munoz

Key Takeaways:

1. Practiced core SQL concepts used in data analytics roles
2. Built multi-table JOIN queries reflecting real business logic
3. Applied aggregate and window functions to real-life scenarios
4. Gained confidence with filtering, grouping, and ranking data
5. Developed subqueries for use cases like identifying gaps or exclusions
6. Strengthened understanding of CASE statements and conditional logic
7. Performed data filtering, NULL handling, and logical categorization
8. Produced clean, actionable insights aligned with stakeholder needs.
9. Effectively used Trello to structure and manage the entire project workflow – from schema design and DML planning to query execution and reporting – reinforcing project management skills essential for analyst roles.

Workflow Breakdown – Project Management:

Day 1: Built the ERD and structured 6 normalized tables with primary and foreign keys. And, executed DDL scripts in MySQL to create a clean, well-related database foundation.

Day 2: Inserted 30-40 realistic records into each table with NULLs, typos, and spacing errors. And created a Trello board to track DDL, DML, query sets, and reporting tasks.

Day 3: Focused on basic SQL syntax, including SELECT, WHERE, JOIN, GROUP BY, and ORDER BY. Solved questions related to customer orders, revenue status, and payment trends.

Day 4: Worked with filters, COALESCE, handling NULLs, and window functions like RANK. Analyzed top-performing cities, payment issues, and revenue segments.

Day 5: Wrote queries using CASE statements, subqueries, and customer/product labelling logic. Targeted returns, customer segments, and price-based product categorization.

Day 6: Reviewed all 15+ queries and aligned them with SQL concepts covered in learning. Ensured Joins, Aggregates, CASE, Subqueries, and Filtering were all demonstrated.

Day 7: Compiled ERD, concepts, question purposes, and business insights into the final report. Prepared export formats (.docx, .pdf, .html) and added project timeline and summary.

Conclusion:

The UrbanSphere SQL project demonstrates practical, entry-level to intermediate SQL capabilities for business analytics. It showcases not only how to write queries but how to translate data into decisions – from spotting high-value customers to identifying payment behavior trends and return issues.